

Hochschule Luzern

Bachelor of Science in Information and Cybersecurity

AI-gestützte Threat Intelligence

ISLAB FS26

16. April 2026

Autoren:

Lenny Johner
Lenny.Johner@stud.hslu.ch

Nik Wey
Nik.Wey@stud.hslu.ch

Coach:

Daniel Frei
Daniel.Frei@hslu.ch

Modulverantwortlicher:

Daniel Frei
Daniel.Frei@hslu.ch

Hinweis

Diese Arbeit wurde ohne Nachbearbeitung durch Dozierende erstellt. Jegliche Veröffentlichung oder Verwendung der Arbeit, auch in Auszügen, ist ohne das ausdrückliche Einverständnis der Studiengangleitung der Hochschule Luzern – Information and Cyber Security nicht gestattet.

Einsatz von Künstlicher Intelligenz

Im Rahmen dieser Arbeit wurde Künstliche Intelligenz eingesetzt, um die Qualität des Textes zu steigern, indem Satzstrukturen optimiert und die sprachliche Klarheit verbessert wurden. Darüber hinaus kamen KI-gestützte Werkzeuge zur Überprüfung von Rechtschreibung und Grammatik zum Einsatz. Falls Künstliche Intelligenz für inhaltliche Aspekte verwendet wurde, ist dies im Quellenverzeichnis unter Angabe des verwendeten Modells sowie des spezifischen Prompts transparent deklariert.

Copyright © 2025 Hochschule Luzern - Informatik

Alle Rechte vorbehalten. Kein Teil dieser Arbeit darf ohne ausdrückliche schriftliche Zustimmung der Studiengangleitung der Hochschule Luzern – Information and Cyber Security in irgendeiner Weise vervielfältigt, veröffentlicht oder in eine maschinenlesbare Sprache umgewandelt werden.

Abstract

Inhaltsverzeichnis

1	Einleitung	1
2	Problemstellung und Zielsetzung	2
2.1	Ausgangslage und Problemstellung	2
2.2	Forschungsziele und Metriken	2
3	Stand der Technik und theoretische Grundlagen	5
3.1	Cyber Threat Intelligence	5
3.1.1	Der Intelligence Cycle	5
3.1.2	Datenformate und Standards (STIX 2.1, TAXII)	5
3.1.3	Threat Intelligence Plattformen (OPENCTI, MISP)	6
3.2	Detektionsmechanismen und SIEM	6
3.2.1	Regelbasierte Detektion (Sigma-Standard)	6
3.2.2	Integration von CTI in Security Operations	7
3.3	Large Language Models im Cybersecurity-Kontext	7
3.3.1	Funktionsweise und Architekturen (Transformer)	7
3.3.2	Prompt Engineering und Fine-Tuning für Security-Daten	8
3.3.3	Autonome Agenten-Systeme und Tool-Use	8
3.3.4	Datensouveränität und Open-Source-LLMs (Ollama)	8
4	Konzeption der automatisierten Cyber-Defense-Pipeline	11
4.1	Anforderungen und Design-Prinzipien	11
4.2	Entwurf der Ingestions- und Parsing-Logik	12
4.3	Architektur des LLM-gestützten Transformations-Agenten	12
4.4	Setup und Komponenten des Proof of Concept	13
4.5	Konzeption des Open-Source-Betriebsmodells	13
5	Methodik	15
5.1	Wissenschaftliche Vorgehensweise	15
5.2	Versuchsaufbau und Test-Szenarien	15
5.3	Durchführung der Messreihen	15
5.4	Metriken zur Erfolgsbewertung	15
6	Realisierung des Automatisierungssystems	16
6.1	Technische Infrastruktur und Toolstack	16
6.1.1	Bereitstellung lokaler Sprachmodelle mittels Ollama	16
6.2	Implementierung der Ingestions-Pipeline	16
6.2.1	Parsing-Logik für unstrukturierte Daten	16
6.2.2	Mapping auf das STIX 2.1-Datenmodell	16
6.3	Entwicklung des LLM-Agenten	16
6.3.1	Prompt-Engineering und Kontext-Management	16
6.3.2	Automatisierte Generierung von Sigma-Regeln	16
6.4	Integration und Schnittstellenkonfiguration	16

7	Evaluation und Ergebnisanalyse	17
7.1	Testumgebung und Datengrundlage	17
7.2	Analyse der Performance-Metriken	17
7.2.1	Durchlaufzeiten und Latenzbetrachtung	17
7.2.2	Effizienzgewinn der automatisierten Regelerstellung	17
7.3	Qualitätsbewertung der generierten Detektionsregeln	17
7.3.1	Syntaktische Validierung der Sigma-Regeln	17
7.3.2	Inhaltliche Präzision und False-Positive-Betrachtung	17
7.4	Diskussion der Ergebnisse und Limitationen	17
8	Ausblick	18
9	Fazit	19
	Abkürzungsverzeichnis	20
	Abbildungsverzeichnis	21
	Tabellenverzeichnis	22
	Gesamtes Literaturverzeichnis	23
	Anhang	24

1 Einleitung

Erklärung was Threat Intelligence ist. Erklärung wie wir das einsetzen wollen Erklärung wie was es bringen soll Erklärung Thema Agenten SOC

2 Problemstellung und Zielsetzung

Das vorliegende Kapitel legt das Fundament für die durchzuführenden Forschungsarbeiten. Um die Relevanz der Arbeit zu begründen, wird zunächst die aktuelle Ausgangslage in der Cyber Defense skizziert und die daraus resultierende Problemstellung abgeleitet. Basierend auf den identifizierten Defiziten bestehender manueller Prozesse werden im zweiten Schritt die konkreten, überprüfbaren Forschungsziele definiert, die zur Lösung des Problems erreicht werden müssen.

2.1 Ausgangslage und Problemstellung

In der modernen Cyber Defense stellt die zeitnahe Verarbeitung von Threat Intelligence (TI) einen kritischen Erfolgsfaktor dar, um auf neuartige Bedrohungen reagieren zu können. In der aktuellen Praxis existiert jedoch ein signifikanter Engpass: Unstrukturierte Bedrohungsdaten müssen häufig manuell analysiert, extrahiert und in maschinenlesbare Formate überführt werden. Eine Untersuchung zur Extraktion von verwertbaren CTI-Informationen aus unstrukturierten Berichten beziffert die durchschnittliche menschliche Arbeitszeit („Human Baseline“) für diesen Schritt auf 41 Minuten pro Bericht (Milenkoski & Cirstea, 2026).

Ebenso erfordert die anschliessende Transformation dieser aufbereiteten Daten in anwendbare Detektionsregeln für Security Information and Event Management (SIEM)-Systeme (z. B. Sigma-Regeln) einen weiteren hohen manuellen Aufwand. Die Erstellung einer einzelnen Regel nimmt durchschnittlich 45 Minuten in Anspruch (Konstantaras et al., 2026). Dieser zweistufige manuelle Prozess ist folglich stark zeitintensiv und skaliert nicht mit der exponentiell wachsenden Menge an Bedrohungsdaten. Daraus resultiert eine stark verzögerte Reaktionszeit (Time-to-Detect), welche potenziellen Angreifern ein kritisches Zeitfenster für ihre Operationen eröffnet.

Darüber hinaus birgt der potenziell naheliegende Einsatz proprietärer, cloudbasierter Large Language Models (LLMs) zur Automatisierung dieser Aufgaben erhebliche Risiken hinsichtlich der Datensouveränität und Informationssicherheit. Wie aktuelle Analysen zum Einsatz von KI in sicherheitskritischen Umgebungen aufzeigen, besteht bei der Nutzung kommerzieller APIs die Gefahr, dass hochsensible Bedrohungsdaten und unternehmensinterne Telemetriedaten an externe Dienstleister abfliessen oder für das Training fremder Modelle verwendet werden (Aditya et al., 2024; European Union Agency for Cybersecurity, 2023). Zusammenfassend fehlt es an einer durchgängigen, skalierbaren und gleichzeitig datenschutzkonformen Automatisierungskette, welche den gesamten Prozess von der Datenaufnahme (Ingestion) bis zur Regelerstellung effizient bewältigt.

2.2 Forschungsziele und Metriken

Um der beschriebenen Problemstellung zu begegnen, ist das primäre Ziel dieser Arbeit der Nachweis, dass die Effizienz der Cyber Defense durch eine durchgängige Automatisierungskette massgeblich gesteigert werden kann. Dabei wird die Hypothese geprüft, dass die Automatisierung von der Ingestion bis zur Detektion manuelle Prozesse in Bezug auf Geschwindigkeit und Skalierbarkeit übertrifft.

Zur strukturierten Validierung dieser Hypothese werden basierend auf der Problemstellung die nachfolgenden drei Teilziele definiert. Um den Kriterien der Überprüfbarkeit und Eindeutigkeit zu entsprechen, sind diese als überprüfbare Endzustände formuliert und zur eindeutigen Referenzierbarkeit im weiteren Verlauf der Arbeit nummeriert:

1. **Z1 – Automatisierte Ingestion ist implementiert:** Eine Pipeline ist vollständig implementiert, welche unstrukturierte Threat Intelligence automatisiert parst und in das Structured Threat Information Expression (STIX)-2.1-Format transformiert. Das Parsing neuer Werte ist mittels eines Open-Source-Large Language Model (LLM) umgesetzt. Die extrahierten Werte sind direkt in Open Cyber Threat Intelligence Platform (OpenCTI) integriert. Der zeitliche Durchlauf der gesamten Pipeline ist in weniger als 15 Minuten abgeschlossen. Dies stellt eine signifikante Effizienzsteigerung gegenüber der manuellen Baseline von 41 Minuten (Milenkoski & Cirstea, 2026) dar.
2. **Z2 – LLM-gestützte Regelerstellung ist etabliert:** Ein Workflow zur automatisierten Transformation von Bedrohungsdaten aus OpenCTI in SIEM-Regeln (z. B. Sigma) ist erfolgreich erstellt. Dabei ist eine messbare Zeitersparnis von mindestens 70 % gegenüber der manuellen Erstellung (Referenzwert: 45 Minuten (Konstantaras et al., 2026)) sowie eine syntaktische Korrektheit der generierten Regeln von mindestens 90 % erreicht.
3. **Z3 – Durchgängiger Open-Source-Ansatz mittels Ollama ist realisiert:** Sämtliche LLM-basierten Komponenten der Automatisierungskette sind als durchgängiger Open-Source-Ansatz umgesetzt. Die zugrundeliegenden Sprachmodelle sind unter der Verwendung von Ollama lokal eingebunden, wodurch proprietäre Abhängigkeiten vermieden und die Datensouveränität im gesamten Verarbeitungsprozess gewährleistet ist. Dies adressiert direkt die identifizierten Datenschutzrisiken cloudbasierter Alternativen (European Union Agency for Cybersecurity., 2023).

Literaturverzeichnis

- Aditya, H., Chawla, S., Dhingra, G., Rai, P., Sood, S., Singh, T., Wase, Z. M., Bahga, A., & Madiseti, V. K. (2024). Evaluating Privacy Leakage and Memorization Attacks on Large Language Models (LLMs) in Generative AI Applications. *Journal of Software Engineering and Applications*, 17(05), 421–447. <https://doi.org/10.4236/jsea.2024.175023>
- European Union Agency for Cybersecurity. (2023). *A multilayer framework for good cybersecurity practices for AI :: security and resilience for smart health services and infrastructures*. Publications Office. Zugriff 13. April 2026 unter <https://data.europa.eu/doi/10.2824/588830>
- Konstantaras, I., Chatzoglou, E., Kampourakis, K. E., & Kambourakis, G. (2026, März). Evaluating LLMs for the Automated Generation of Operational Detection Rules in Enterprise EDR Environments. <https://doi.org/10.20944/preprints202603.1994.v1>
- Milenkoski, A., & Cirstea, R. G. (2026, März). From Narrative to Knowledge Graph — LLM-Driven Information Extraction in Cyber Threat Intelligence. Zugriff 13. April 2026 unter <https://www.sentinelone.com/labs/from-narrative-to-knowledge-graph-llm-driven-information-extraction-in-cyber-threat-intelligence/>

3 Stand der Technik und theoretische Grundlagen

Um die in dieser Arbeit konzipierte Automatisierungskette fundiert entwickeln und bewerten zu können, bedarf es eines soliden Verständnisses der zugrundeliegenden Konzepte und Technologien. Das vorliegende Kapitel beleuchtet daher den aktuellen Stand der Technik sowie die theoretischen Grundlagen. Zunächst wird das Konzept der Cyber Threat Intelligence (CTI) mit seinen Prozessen, Formaten und Plattformen eingeführt. Daran anknüpfend werden im weiteren Verlauf der Arbeit die etablierten Detektionsmechanismen sowie die Einsatzpotenziale von Large Language Models im Cybersecurity-Umfeld betrachtet.

3.1 Cyber Threat Intelligence

Die proaktive Abwehr von Cyberangriffen erfordert detailliertes Wissen über die Taktiken, Techniken und Prozeduren potenzieller Angreifer. Dieses aufbereitete Wissen wird unter dem Begriff CTI zusammengefasst. Um rohe Bedrohungsdaten in verwertbare Informationen zu überführen und technisch nutzbar zu machen, haben sich spezifische Vorgehensweisen und Standards etabliert. Der folgende Abschnitt erläutert zunächst den prozessualen Ablauf der Informationsgewinnung anhand des Intelligence Cycles. Darauf aufbauend werden die vorherrschenden Datenformate für den maschinellen Austausch vorgestellt, bevor abschliessend gängige Plattformen betrachtet werden, die zur zentralen Speicherung dieser Daten dienen.

3.1.1 Der Intelligence Cycle

Um aus einer unstrukturierten Menge an Bedrohungsdaten verwertbare Erkenntnisse zu generieren, wird in der Praxis der sogenannte Intelligence Cycle angewendet. Dieser zirkuläre Prozess stellt sicher, dass die gesammelten Informationen systematisch aufbereitet und auf die strategischen sowie operativen Ziele einer Organisation ausgerichtet werden. Der klassische Intelligence Cycle gliedert sich in der Regel in sechs Phasen: Planung und Richtungsweisung (Direction), Datensammlung (Collection), Verarbeitung (Processing), Analyse und Produktion (Analysis), Verteilung (Dissemination) sowie Feedback („ENISA Threat Landscape Methodology — ENISA“, 2025; Heuer, 1999).

Für die vorliegende Arbeit ist insbesondere die Phase der Verarbeitung (Processing) von zentraler Bedeutung. In dieser Phase müssen roh gesammelte Daten geparkt, normalisiert und in maschinenlesbare Formate überführt werden – ein ressourcenintensiver Schritt, der den primären Ansatzpunkt für die Konzeption der in dieser Arbeit entwickelten Automatisierungslösung darstellt.

3.1.2 Datenformate und Standards (STIX 2.1, TAXII)

Für den effizienten und automatisierten Austausch von CTI ist eine Standardisierung der Datenstrukturen unerlässlich. Als De-facto-Standard hat sich hierbei STIX etabliert,

welches vom OASIS-Konsortium in der aktuellen Version 2.1 spezifiziert wird (Bret et al., 2021). Im Gegensatz zu älteren, primär listenbasierten Formaten nutzt STIX 2.1 eine graphenbasierte Architektur. Entitäten wie Angreifer, Malware oder Kampagnen werden als STIX Domain Objects (SDO) modelliert, während die Beziehungen zwischen ihnen durch STIX Relationship Objects (SRO) abgebildet werden.

Ergänzend zu STIX definiert der Standard Trusted Automated Exchange of Intelligence Information (TAXII) das zugehörige Transportprotokoll (Bret & Drew, 2021). TAXII spezifiziert RESTful Application Programming Interface (API) sowie die HTTPS-basierte Kommunikation, um den sicheren Transfer von STIX-formatierten Daten zwischen verschiedenen Systemen und Organisationen zu gewährleisten. Die Kombination beider Standards ermöglicht somit eine semantisch reiche und maschinenlesbare Repräsentation von Bedrohungsszenarien.

3.1.3 Threat Intelligence Plattformen (OPENCTI, MISP)

Zur Aggregation, Speicherung und Verwaltung der formatierten CTI-Daten kommen spezialisierte Threat Intelligence Plattformen (TIP) zum Einsatz. Zu den am weitesten verbreiteten Open-Source-Lösungen zählen die Malware Information Sharing Platform (MISP) sowie die OpenCTI.

Während MISP historisch stark auf den schnellen Austausch von einzelnen Indikatoren (Indicator of Compromise (IoC)) fokussiert ist (Wagner et al., 2016), bietet OpenCTI eine vollumfängliche, graphenbasierte Datenbankstruktur. OpenCTI ist nativ auf das STIX-2.1-Datenmodell ausgerichtet und ermöglicht es, komplexe Zusammenhänge zwischen Bedrohungsakteuren und technischen Indikatoren als Knowledge Graph zu visualisieren und maschinell zu verarbeiten („OpenCTI Documentation“, o. D.). Aufgrund dieser nativen STIX-Kompatibilität und der umfassenden API-Schnittstellen bildet OpenCTI in der vorliegenden Arbeit das zentrale Element zur Speicherung der automatisiert ingestierten Threat Intelligence.

3.2 Detektionsmechanismen und SIEM

Wie im vorangegangenen Abschnitt dargelegt, liefert CTI das notwendige Wissen über potenziell gefährliche Akteure und deren Methoden. Um dieses Wissen jedoch operativ nutzbar zu machen, bedarf es technischer Systeme, die Sicherheitsvorfälle in IT-Infrastrukturen automatisiert erkennen und melden. Security Information and Event Management (SIEM) bildet hierbei das technologische Rückgrat moderner Security Operations Center (Security Operations Center (SOC)). Der folgende Abschnitt erläutert zunächst die Funktionsweise der regelbasierten Detektion unter besonderer Berücksichtigung des Sigma-Standards. Daran anschliessend wird die prozessuale und technische Integration von CTI in den operativen SIEM-Betrieb analysiert, um den Übergang von der theoretischen Bedrohungslage zur aktiven Abwehr aufzuzeigen.

3.2.1 Regelbasierte Detektion (Sigma-Standard)

Zentrale Kernkomponente eines SIEM-Systems ist die Korrelations- und Detektions-Engine, welche aggregierte Logdaten von Endpunkten, Netzwerken und Cloud-Diensten kontinuierlich gegen vordefinierte Regelwerke abgleicht. Traditionell sind diese Detektionsregeln proprietär und eng an die spezifische Abfragesprache des jeweiligen SIEM-Herstellers (wie Splunk SPL oder Microsoft KQL) gebunden. Um diesem Vendor-Lock-in entgegenzuwirken und eine herstellerunabhängige Beschreibung von Log-Ereignissen zu ermöglichen,

hat sich der Sigma-Standard als generisches und offenes Signaturformat etabliert („SigmaHQ/sigma“, 2026).

Sigma basiert auf einer strukturierten, menschenlesbaren YAML-Syntax und abstrahiert die Detektionslogik von der zugrundeliegenden technologischen Infrastruktur. Ähnlich wie YARA für die dateibasierte Malware-Erkennung oder Snort für den Netzwerkverkehr, fungiert Sigma als universelle Sprache für Logdaten.

Über sogenannte Sigma-Konverter (Backends) lassen sich die generischen YAML-Regeln automatisiert in spezifische Abfragesprachen übersetzen, wodurch eine Regel in verschiedenen SIEM-Umgebungen eingesetzt werden kann. In der vorliegenden Arbeit dient Sigma aufgrund dieser Interoperabilität als definiertes Zielformat für den LLM-gestützten Übersetzungsagenten, um aus rohen Bedrohungsdaten anwendbare und systemübergreifende Detektionslogik zu generieren.

3.2.2 Integration von CTI in Security Operations

Die isolierte Bereitstellung eines SIEM-Systems und einer CTI-Plattform wie OpenCTI reicht nicht aus, um eine effektive Cyber Defense zu gewährleisten; entscheidend ist die nahtlose und zeitnahe Integration beider Domänen. Diese sogenannte Operationalisierung von CTI beschreibt den Prozess, bei dem strategische und taktische Bedrohungsinformationen in anwendbare, technische Detektionsmechanismen für das SOC überführt werden (Kotsias et al., 2023).

In der traditionellen Praxis erfordert dieser Brückenschlag einen erheblichen manuellen Analyseaufwand: Security-Analysten müssen unstrukturierte Threat-Reports lesen, relevante Verhaltensmuster oder STIX-Daten extrahieren und diese manuell in Sigma- oder herstellerspezifische Regeln umwandeln. Wie in der Problemstellung (Kapitel 2) dargelegt, stellt dieser manuelle Transformationsprozess einen massiven zeitlichen Flaschenhals dar. Genau an dieser Schnittstelle zwischen der CTI-Verwaltung und der aktiven Detektion im SIEM setzt die im Rahmen dieser Arbeit konzipierte Automatisierungspipeline an. Ziel ist es, den fehleranfälligen und verzögerten Übertragungszyklus durch den Einsatz von generativer Künstlicher Intelligenz zu überwinden und eine maschinengetriebene, skalierbare Integration von CTI in Security Operations zu etablieren.

3.3 Large Language Models im Cybersecurity-Kontext

Die in den vorangegangenen Abschnitten beschriebene manuelle Extraktion von CTI sowie deren Überführung in SIEM-Detektionsregeln stellt einen massiven zeitlichen Engpass dar. Um diesen Prozess durchgängig zu automatisieren, rücken Large Language Models (LLMs) zunehmend in den Fokus der Cybersecurity-Forschung. Diese Modelle besitzen die Fähigkeit, unstrukturierte Textdaten (wie Threat-Reports) semantisch zu erfassen und in strukturierte Code- oder Datenformate zu übersetzen (Hanxiang Xu et al., 2024). Der folgende Abschnitt beleuchtet zunächst die zugrundeliegende Architektur dieser Modelle. Darauf aufbauend werden Techniken zur Domänenadaption sowie der Einsatz autonomer Agenten-Systeme erläutert. Abschliessend wird die für diese Arbeit zentrale Problematik der Datensouveränität adressiert und der Lösungsansatz mittels lokaler Open-Source-LLMs (Ollama) begründet.

3.3.1 Funktionsweise und Architekturen (Transformer)

Die technologische Basis moderner Large Language Models bildet die sogenannte Transformer-Architektur, welche 2017 von Vaswani et al. eingeführt wurde (Vaswani et al., 2023). Im

Gegensatz zu früheren rekurrenten neuronalen Netzen (RNNs), die Texte streng sequenziell verarbeiteten, ermöglicht der Transformer eine parallele Verarbeitung grosser Datenmengen.

Das Kernelement dieser Architektur ist der *Self-Attention-Mechanismus* (Selbstaufmerksamkeit). Dieser mathematische Prozess berechnet für jedes Token (Wortbruchstück) in einer Eingabesequenz, wie stark es mit allen anderen Token im Kontext korreliert. Im Cybersecurity-Kontext ist diese Eigenschaft von immenser Bedeutung: Wenn ein Modell einen langen Threat-Report analysiert, ermöglicht die Self-Attention, weit auseinanderliegende technische Entitäten (wie etwa eine IP-Adresse im ersten Absatz und den dazugehörigen Malware-Namen im letzten Absatz) sicher miteinander in Beziehung zu setzen und als zusammengehöriges Muster zu erkennen (Vaswani et al., 2023).

3.3.2 Prompt Engineering und Fine-Tuning für Security-Daten

Obwohl vortrainierte Basis-LLMs über ein breites Allgemeinwissen verfügen, benötigen sie für hochspezifische Aufgabenstellungen – wie das korrekte Parsing von STIX-Objekten oder die fehlerfreie Syntax-Generierung von Sigma-Regeln – eine gezielte Domänenadaptation. Die gängigste und ressourceneffizienteste Methode hierfür ist das Prompt Engineering, insbesondere das *In-Context Learning*.

Hierbei wird das Modell nicht durch ein aufwendiges Training seiner Gewichte (Fine-Tuning) verändert. Stattdessen werden der Eingabeaufforderung (Prompt) hochdetaillierte Instruktionen, JSON-Schemata sowie konkrete Lösungsbeispiele (Few-Shot Prompting) mitgegeben. Xu et al. (Hanxiang Xu et al., 2024) belegen in ihrer Literaturrecherche, dass LLMs durch präzise kontextuelle Instruktionen signifikant weniger halluzinieren und ihre Genauigkeit bei sicherheitsspezifischen Extraktionsaufgaben drastisch steigt. Für die in dieser Arbeit implementierte Pipeline bildet strukturiertes Prompt Engineering die methodische Grundlage, um rohe CTI-Daten formattreu zu verarbeiten.

3.3.3 Autonome Agenten-Systeme und Tool-Use

Die blosse Textgenerierung reicht für eine durchgängige Automatisierung von der Ingestion bis zur SIEM-Regel nicht aus. Das LLM muss vielmehr aktiv mit Umsystemen interagieren (z. B. der OpenCTI-API). Dies wird durch das Paradigma der autonomen LLM-Agenten realisiert. Ein etablierter Ansatz hierfür ist das ReAct-Framework (Reasoning and Acting), das von Yao et al. spezifiziert wurde (S. Yao et al., 2023).

ReAct erweitert das Modell um eine kognitive Schleife: Das LLM "denkt" zunächst über das Problem nach (Reasoning), leitet daraus eine konkrete Handlung ab (Acting, z. B. den API-Aufruf zum Speichern eines STIX-Objekts), wertet die Rückmeldung des Systems aus (Observation) und passt seinen Plan entsprechend an. Durch diesen sogenannten Tool-Use wird das Sprachmodell von einem passiven Übersetzer zu einem aktiven Orchestrator der Automatisierungskette.

3.3.4 Datensouveränität und Open-Source-LLMs (Ollama)

Trotz ihrer technologischen Leistungsfähigkeit birgt die Nutzung proprietärer Cloud-LLMs (wie OpenAI GPT-4) im Cybersecurity-Sektor gravierende Risiken. Die zu verarbeitenden Threat-Reports enthalten häufig hochsensible Telemetriedaten, interne Netzwerktopologien oder Zero-Day-Vektoren. Ein Transfer dieser Daten an externe API-Dienstleister verletzt grundlegende Prinzipien der Datensouveränität (Privacy Leakage) und birgt die

Gefahr, dass die Daten für das Training fremder Modelle missbraucht werden (Y. Yao et al., 2024).

Um diesem Problem zu begegnen, setzt diese Arbeit auf einen streng lokalen Open-Source-Ansatz (Ziel Z3). Durch die Nutzung lokal ausführbarer Modelle (Open-Weights), betrieben über das Framework Ollama, verlassen zu keinem Zeitpunkt sensible CTI-Daten die kontrollierte Infrastruktur. Dieser Ansatz vereint die kognitiven Fähigkeiten moderner LLMs mit den strikten Compliance- und Geheimhaltungsanforderungen eines modernen Security Operations Centers.

Literaturverzeichnis

- Bret, J., & Drew, V. (2021, Juni). TAXII™ Version 2.1. <https://docs.oasis-open.org/cti/taxii/v2.1/os/taxii-v2.1-os.html>
- Bret, J., Rich, P., & Trey, D. (2021, Januar). STIX™ Version 2.1. <https://docs.oasis-open.org/cti/stix/v2.1/cs02/stix-v2.1-cs02.html>
- ENISA Threat Landscape Methodology — ENISA. (2025, November). Zugriff 14. April 2026 unter <https://www.enisa.europa.eu/publications/enisa-threat-landscape-methodology>
- Hanxiang Xu, Shenao Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Chen, K., Yu, T., Liu, Y., & Haoyu Wang. (2024). Large Language Models for Cyber Security: A Systematic Literature Review. <https://doi.org/10.13140/RG.2.2.10132.92800>
- Heuer, R. J. (1999). *Psychology of intelligence analysis*. Center for the Study of Intelligence, Central Intelligence Agency.
- Kotsias, J., Ahmad, A., & Scheepers, R. (2023). Adopting and integrating cyber-threat intelligence in a commercial organisation. *European Journal of Information Systems*, 32(1), 35–51. <https://doi.org/10.1080/0960085X.2022.2088414>
- OpenCTI Documentation. (o. D.). Zugriff 14. April 2026 unter <https://docs.opencti.io/7.260401.0/>
- SigmaHQ/sigma [original-date: 2016-12-24T09:48:49Z]. (2026, April). Zugriff 14. April 2026 unter <https://github.com/SigmaHQ/sigma>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023, August). Attention Is All You Need [arXiv:1706.03762]. <https://doi.org/10.48550/arXiv.1706.03762>
- Wagner, C., Dulaunoy, A., Wagener, G., & Iklody, A. (2016). MISP: The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform. *Proceedings of the 2016 ACM on Workshop on Information Sharing and Collaborative Security*, 49–56. <https://doi.org/10.1145/2994539.2994542>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023, März). ReAct: Synergizing Reasoning and Acting in Language Models [arXiv:2210.03629]. <https://doi.org/10.48550/arXiv.2210.03629>
- Yao, Y., Duan, J., Xu, K., Cai, Y., Sun, Z., & Zhang, Y. (2024). A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly. *High-Confidence Computing*, 4(2), 100211. <https://doi.org/10.1016/j.hcc.2024.100211>

4 Konzeption der automatisierten Cyber-Defense-Pipeline

Das vorliegende Kapitel überführt die formulierten Forschungsziele sowie die theoretischen Grundlagen in ein fundiertes technisches Systemkonzept. In Anlehnung an das Paradigma der Design Science Research (Hevner et al., 2004) wird dabei ein strukturiertes Artefakt entworfen, welches die identifizierten Probleme der manuellen Threat-Intelligence-Verarbeitung adressiert. Zu diesem Zweck werden zunächst die funktionalen und qualitativen Anforderungen aus der Zielsetzung deduziert sowie übergeordnete Design-Prinzipien definiert. Darauf aufbauend erfolgen die detaillierte Konzeption der Ingestions-Logik, der Entwurf der Agenten-Architektur zur Regelerstellung sowie die Planung des sicheren Betriebsmodells.

4.1 Anforderungen und Design-Prinzipien

Um eine systematische und zielgerichtete Entwicklung der Automatisierungspipeline zu gewährleisten, müssen die definierten Projektziele (Z1 bis Z3) in konkrete, überprüfbare Systemanforderungen (Requirements) übersetzt werden. Gemäß etablierten Methoden des Requirements Engineerings (Pohl, 2010) gliedern sich diese primär in funktionale Anforderungen (Functional Requirements), qualitative Systemmerkmale (Performance und Security) sowie architektonische Rahmenbedingungen.

Funktionale Anforderungen und Interoperabilität

Aus dem Ziel der automatisierten Ingestion (Z1) leitet sich die primäre funktionale Anforderung ab, dass das System in der Lage sein muss, unstrukturierte Textberichte selbstständig aufzunehmen und semantisch zu parsen. Um den nahtlosen Informationsaustausch zwischen verschiedenen Sicherheitswerkzeugen zu garantieren, muss die Pipeline die extrahierten Informationen zwingend auf das STIX-2.1-Format mappen. Wie Silva et al. in ihrer umfassenden Evaluation von Cyber-Threat-Intelligence-Standards belegen (De Melo E Silva et al., 2020), ist STIX 2.1 die mit Abstand fähigste Taxonomie, um fragmentiertes Wissen systemübergreifend maschinenlesbar zu machen. Die strukturierten Daten müssen anschließend automatisiert über entsprechende API-Schnittstellen in die OpenCTI-Plattform persistiert werden.

Zusätzlich fordert Ziel Z2 die funktionale Fähigkeit des Systems, aus diesen nun strukturiert vorliegenden OpenCTI-Daten anwendbare SIEM-Regeln zu generieren. Das Systemkonzept muss folglich einen LLM-basierten Übersetzungs-Workflow vorsehen, der das generische YAML-Format des Sigma-Standards als standardisierte Ausgabesprache nutzt.

Performance- und Qualitätsanforderungen

Neben der reinen Funktionalität unterliegt das System harten nicht-funktionalen Leistungsanforderungen. Die Konzeption muss eine asynchrone und ressourcenoptimierte Verarbeitung sicherstellen, um den zeitlichen Durchlauf der gesamten Ingestions-Pipeline (Z1) auf unter 15 Minuten zu begrenzen.

Ebenso muss das Design des Transformations-Workflows (Z2) so optimiert werden – beispielsweise durch strukturiertes Prompt Engineering –, dass eine Zeitersparnis von min-

destens 70 % gegenüber der manuellen Regelerstellung erreicht wird. Darüber hinaus unterliegt die Ausgabe einer strengen Qualitätsmetrik: Da fehlerhafte Regeln potenziell den Ausfall der Detektions-Engine oder massive False-Positive-Raten im Security Operations Center auslösen können, ist eine syntaktische Korrektheit der generierten SIEM-Regeln von mindestens 90 % zwingend in das Design der Pipeline zu integrieren.

Design-Prinzipien: Datensouveränität und Security-by-Design

Die elementarste architektonische Rahmenbedingung der vorliegenden Arbeit entspringt dem Ziel Z3. Das System muss nach dem Prinzip des *Security by Design* konzipiert werden. Wie Bygrave in seiner regulatorischen Analyse aufzeigt (Bygrave, 2022), zwingt dieses Prinzip Software-Architekten dazu, Sicherheitsbedürfnisse und Datenschutzvorgaben (Privacy) nicht erst nachträglich (als Patch“), sondern bereits in der initialen Konstruktionsphase eines Systems tief zu verankern.

Da in CTI-Berichten hochsensible Angriffsvektoren verarbeitet werden, schließt das Design die Nutzung kommerzieller, cloudbasierter KI-Schnittstellen kategorisch aus, um Privacy Leakage und unkontrollierten Datenabfluss zu verhindern (Y. Yao et al., 2024). Das Kernprinzip der Architektur ist folglich ein durchgängiger Open-Source-Ansatz. Das Konzept sieht vor, sämtliche analytischen Prozesse mittels offener Sprachmodelle durchzuführen, welche lokal über das Framework Ollama instanziiert werden. Diese strikte Isolierung gewährleistet die absolute Datensouveränität (Z3) während des gesamten Verarbeitungsprozesses.

4.2 Entwurf der Ingestions- und Parsing-Logik

Der Entwurf der Ingestions-Logik folgt einem dreiphasigen Modell: Sanitization, Extraktion und Normalisierung (Milenkoski & Cirstea, 2026). Im ersten Schritt werden unstrukturierte Berichte (z. B. PDF- oder HTML-Dokumente) mittels einer Konvertierungskomponente in bereinigten Plain Text überführt.

Die semantische Extraktion erfolgt durch spezialisierte Agenten, die mittels *Few-Shot Prompting* darauf trainiert sind, Entitäten wie Bedrohungsakteure, Malware-Varianzen und technische Indikatoren (IoCs) zu identifizieren. Um die Fehlerraten (Halluzinationen) zu minimieren, sieht das Konzept eine Extraktionsprüfung gegen das offizielle STIX-2.1-Vokabular vor. Der finale Prozessschritt übersetzt diese extrahierten Fakten deterministisch in STIX-Objekte und deren Relationen (SROs), bevor die Persistierung in OpenCTI über die Client-API erfolgt (Madisetti, 2025).

4.3 Architektur des LLM-gestützten Transformations-Agenten

Die Transformation von strukturierten CTI-Daten in anwendbare Sigma-Regeln basiert auf einem agentischen Workflow nach dem *ReAct*-Paradigma (Reasoning and Acting) (S. Yao et al., 2023). Der Agent fungiert hierbei als kognitiver Kern, der iterativ mit der OpenCTI-Datenbank interagiert.

Der Prozess umfasst drei Sub-Module:

- **Context Retrieval:** Der Agent fragt über eine RAG-Schnittstelle (Retrieval-Augmented Generation) kontextrelevante Sigma-Templates ab, die auf die identifizierten MITRE ATT&CK Techniken gemappt sind (Konstantaras et al., 2026).
- **Synthese:** Das lokale LLM generiert die YAML-Logik der Regel unter Berücksichtigung spezifischer Log-Quellen (z. B. Sysmon oder Windows Event Logs).

- **Validation-Loop:** Ein sekundärer Prozess prüft die syntaktische Validität der generierten Regel gegen das Sigma-Schema („SigmaHQ/sigma“, 2026). Bei Fehlern wird ein Feedback-Loop an das Modell initiiert (Self-Correction).

4.4 Setup und Komponenten des Proof of Concept

Die technische Realisierung des Proof of Concept (PoC) basiert auf einer Container-Architektur (Docker). Als Inferenz-Engine dient *Ollama*, da es eine einfache Steuerung lokaler Modelle bei gleichzeitig hoher Performance bietet („library“, o. D.). Die Orchestrierung übernimmt *LangChain*, um die Schnittstellen zwischen der OpenCTI-Client-Library und dem LLM-Endpunkt zu bedienen.

4.5 Konzeption des Open-Source-Betriebsmodells

Das Betriebsmodell ist auf maximale Datensouveränität ausgelegt. Durch die Nutzung lokaler Modelle entfallen variable Token-Kosten und regulatorische Hürden bezüglich der Verarbeitung sensibler Bedrohungsdaten. Die Wartung der Pipeline erfolgt durch SOC-Analysten (Security Operations Center), wobei ein *Human-in-the-Loop*-Ansatz verfolgt wird: Generierte Detektionsregeln werden erst nach einer manuellen Review-Phase produktiv geschaltet. Dies gewährleistet, dass die KI als Assistenzsystem fungiert, während die finale Entscheidungshoheit beim Sicherheitspersonal verbleibt.

Anhang: CRAAP-Test für Quellen ohne DOI

1. Ollama Team (2024): Ollama Documentation

- **Currency:** Die Dokumentation wird tagesaktuell gepflegt, passend zu den Modell-Releases.
- **Relevance:** Es ist die Primärquelle für die technische Implementierung der lokalen Inferenz-Engine.
- **Authority:** Offizielle Publikation der Entwickler des Frameworks; weite Verbreitung in der Industrie.
- **Accuracy:** Die technischen Befehle und Parameter sind durch die funktionierende Software verifizierbar.
- **Purpose:** Technische Instruktion ohne kommerzielle Marketing-Verzerrung.
- **Ergebnis:** Als technische Primärquelle für den PoC geeignet.

2. SigmaHQ (2024): Sigma Specification

- **Currency:** Laufende Weiterentwicklung (Version 1.0+); Industriestandard.
- **Relevance:** Definiert das Zielformat für die automatisierte Regelerstellung (Z2).
- **Authority:** Getragen von einer globalen Community und Experten wie Florian Roth.
- **Accuracy:** Technisch präzise YAML-Schemata, die weltweit in SIEM-Systemen produktiv eingesetzt werden.
- **Purpose:** Standardisierung der Cyber-Abwehr.
- **Ergebnis:** Als Referenzstandard für die Konzeption hochgradig valide.

5 Methodik

5.1 Wissenschaftliche Vorgehensweise

In diesem Abschnitt wird das methodische Vorgehen bei der Entwicklung und Validierung des Proof-of-Concept beschrieben.

5.2 Versuchsaufbau und Test-Szenarien

Um die Leistungsfähigkeit der Automatisierungskette zu prüfen, werden unstrukturierte Berichte aus den Quellen X, Y und Z als Testdatensatz definiert.

5.3 Durchführung der Messreihen

Die Evaluation erfolgt durch einen systematischen Vergleich zwischen dem automatisierten Workflow und einem manuellen Referenzprozess.

5.4 Metriken zur Erfolgsbewertung

Die Bewertung der Ergebnisse stützt sich auf die in Kapitel 1 definierten Key Performance Indicator (KPI)f:

- Durchlaufzeit (Latenz)
- Zeitersparnis-Faktor
- Syntaktische Validitätsrate

6 Realisierung des Automatisierungssystems

6.1 Technische Infrastruktur und Toolstack

6.1.1 Bereitstellung lokaler Sprachmodelle mittels Ollama

6.2 Implementierung der Ingestions-Pipeline

6.2.1 Parsing-Logik für unstrukturierte Daten

6.2.2 Mapping auf das STIX 2.1-Datenmodell

6.3 Entwicklung des LLM-Agenten

6.3.1 Prompt-Engineering und Kontext-Management

6.3.2 Automatisierte Generierung von Sigma-Regeln

6.4 Integration und Schnittstellenkonfiguration

7 Evaluation und Ergebnisanalyse

7.1 Testumgebung und Datengrundlage

7.2 Analyse der Performance-Metriken

7.2.1 Durchlaufzeiten und Latenzbetrachtung

7.2.2 Effizienzgewinn der automatisierten Regelerstellung

7.3 Qualitätsbewertung der generierten Detektionsregeln

7.3.1 Syntaktische Validierung der Sigma-Regeln

7.3.2 Inhaltliche Präzision und False-Positive-Betrachtung

7.4 Diskussion der Ergebnisse und Limitationen

8 Ausblick

Prognosen für die Zukunft was kann man mit unseren ergebnissen anfangen

9 Fazit

Abkürzungsverzeichnis

API Application Programming Interface. 6, 11

CTI Cyber Threat Intelligence. 5–9, 12

IoC Indicator of Compromise. 6

KPI Key Performance Indicator. 15

LLM Large Language Model. 3, 7, 11

MISP Malware Information Sharing Platform. 6

OpenCTI Open Cyber Threat Intelligence Platform. 3, 6–8, 11–13

SDO STIX Domain Objects. 6

SIEM Security Information and Event Management. 2, 3, 6, 7, 11, 12

SOC Security Operations Center. 6, 7

SRO STIX Relationship Objects. 6

STIX Structured Threat Information Expression. 3, 5–8, 11

TAXII Trusted Automated Exchange of Intelligence Information. 6

TIP Threat Intelligence Plattformen. 6

Abbildungsverzeichnis

Tabellenverzeichnis

Gesamtes Literaturverzeichnis

- Aditya, H., Chawla, S., Dhingra, G., Rai, P., Sood, S., Singh, T., Wase, Z. M., Bahga, A., & Madiseti, V. K. (2024). Evaluating Privacy Leakage and Memorization Attacks on Large Language Models (LLMs) in Generative AI Applications. *Journal of Software Engineering and Applications*, 17(05), 421–447. <https://doi.org/10.4236/jsea.2024.175023>
- Bret, J., & Drew, V. (2021, Juni). TAXII™ Version 2.1. <https://docs.oasis-open.org/cti/taxii/v2.1/os/taxii-v2.1-os.html>
- Bret, J., Rich, P., & Trey, D. (2021, Januar). STIX™ Version 2.1. <https://docs.oasis-open.org/cti/stix/v2.1/cs02/stix-v2.1-cs02.html>
- Bygrave, L. A. (2022). Security by Design: Aspirations and Realities in a Regulatory Context. *Oslo Law Review*, 8(3), 126–177. <https://doi.org/10.18261/olr.8.3.2>
- De Melo E Silva, A., Costa Gondim, J. J., De Oliveira Albuquerque, R., & García Villalba, L. J. (2020). A Methodology to Evaluate Standards and Platforms within Cyber Threat Intelligence. *Future Internet*, 12(6), 108. <https://doi.org/10.3390/fi12060108>
- ENISA Threat Landscape Methodology — ENISA. (2025, November). Zugriff 14. April 2026 unter <https://www.enisa.europa.eu/publications/enisa-threat-landscape-methodology>
- European Union Agency for Cybersecurity. (2023). *A multilayer framework for good cybersecurity practices for AI :: security and resilience for smart health services and infrastructures*. Publications Office. Zugriff 13. April 2026 unter <https://data.europa.eu/doi/10.2824/588830>
- Hanxiang Xu, Shenao Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Chen, K., Yu, T., Liu, Y., & Haoyu Wang. (2024). Large Language Models for Cyber Security: A Systematic Literature Review. <https://doi.org/10.13140/RG.2.2.10132.92800>
- Heuer, R. J. (1999). *Psychology of intelligence analysis*. Center for the Study of Intelligence, Central Intelligence Agency.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research1. *MIS Quarterly*, 28(1), 75–106. <https://doi.org/10.2307/25148625>
- Konstantaras, I., Chatzoglou, E., Kampourakis, K. E., & Kambourakis, G. (2026, März). Evaluating LLMs for the Automated Generation of Operational Detection Rules in Enterprise EDR Environments. <https://doi.org/10.20944/preprints202603.1994.v1>
- Kotsias, J., Ahmad, A., & Scheepers, R. (2023). Adopting and integrating cyber-threat intelligence in a commercial organisation. *European Journal of Information Systems*, 32(1), 35–51. <https://doi.org/10.1080/0960085X.2022.2088414>
- library. (o.D.). Zugriff 15. April 2026 unter <https://ollama.com/library>
- Madiseti, V. K. (2025). STIXAgent—A Multi-Agent Framework for Standardized Management of Cyber Threat Intelligence (CTI) Reports. *Journal of Information Security*, 16(04), 544–567. <https://doi.org/10.4236/jis.2025.164028>
- Milenkoski, A., & Cirstea, R. G. (2026, März). From Narrative to Knowledge Graph — LLM-Driven Information Extraction in Cyber Threat Intelligence. Zugriff 13. April

- 2026 unter <https://www.sentinelone.com/labs/from-narrative-to-knowledge-graph-llm-driven-information-extraction-in-cyber-threat-intelligence/>
- OpenCTI Documentation. (o. D.). Zugriff 14. April 2026 unter <https://docs.opencti.io/7.260401.0/>
- Pohl, K. (2010). *Requirements Engineering*. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-642-12578-2>
- SigmaHQ/sigma [original-date: 2016-12-24T09:48:49Z]. (2026, April). Zugriff 14. April 2026 unter <https://github.com/SigmaHQ/sigma>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023, August). Attention Is All You Need [arXiv:1706.03762]. <https://doi.org/10.48550/arXiv.1706.03762>
- Wagner, C., Dulaunoy, A., Wagener, G., & Iklody, A. (2016). MISP: The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform. *Proceedings of the 2016 ACM on Workshop on Information Sharing and Collaborative Security*, 49–56. <https://doi.org/10.1145/2994539.2994542>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023, März). ReAct: Synergizing Reasoning and Acting in Language Models [arXiv:2210.03629]. <https://doi.org/10.48550/arXiv.2210.03629>
- Yao, Y., Duan, J., Xu, K., Cai, Y., Sun, Z., & Zhang, Y. (2024). A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly. *High-Confidence Computing*, 4(2), 100211. <https://doi.org/10.1016/j.hcc.2024.100211>

CRAAP-Test Quelle (Aditya et al., 2024)